

CSE 461 — Programming Assignment 1

A Simple Recursive Ray Tracer

Student: Cemil Fatih Yol **Student ID:** 220104004004 **Date:** April 20, 2026

1. Introduction

This project implements a simple recursive ray tracer in C++17, following the assignment specification. The renderer reads a scene from an XML file, casts rays from a virtual camera through every pixel of the image plane, finds the closest triangle hit, and computes the pixel's color using the Phong shading model with shadows, recursive mirror reflections, and texture mapping. Both bonus features — a hierarchical acceleration structure (BVH) and multithreading — are implemented and benchmarked.

The renderer runs on macOS (tested with Apple Clang 17.0.0 on x86_64) and is built with a plain Makefile. External dependencies are limited to two header-only libraries: TinyXML-2 for XML parsing and stb_image / stb_image_write for image I/O — both within what the assignment explicitly permits.

2. Design Overview

2.1 File layout

```
raytracer/  
  include/  
    Vec3.h          - 3D vector, dot/cross/normalize, color ops  
    Ray.h           - origin + direction + recursion depth  
    Scene.h         - Camera, Material, Light, Face, Mesh, Scene  
    SceneLoader.h   - XML parser interface  
    Renderer.h      - render entry point  
    BVH.h           - AABB, BVHNode, BVH build/traversal  
  src/  
    main.cpp        - CLI parsing, scene load, render dispatch  
    SceneLoader.cpp - TinyXML-2 based parser  
    Renderer.cpp    - camera rays, shading, recursion, threading  
    BVH.cpp         - top-down median-split BVH builder  
    stb_imp1.cpp    - single-translation-unit stb implementations  
  external/  
    tinyxml2.{h,cpp}  
    stb_image.h  
    stb_image_write.h
```

```

scenes/          - XML scenes, OBJ models, textures
Makefile

```

2.2 Data flow

`main` parses CLI flags, calls `loadScene` to parse XML into a `Scene` struct, then calls `render`. The `Scene` is a flat container of vertex / normal / UV pools that mesh faces reference by index — matching the OBJ-style `v/t/n` convention the assignment uses.

`render` builds a BVH over all triangles, dispatches a thread pool that races over image rows via an atomic counter, and writes the result as a PNG. Each thread independently computes pixel colors by calling `rayColor`, which calls `shade` and recurses for mirror reflection. No shared mutable state exists during rendering: each thread writes to disjoint pixel ranges, so no locking is needed.

3. Algorithms Implemented

3.1 Ray-triangle intersection (Möller-Trumbore)

For a ray $p(t) = o + t \cdot d$ and a triangle (v_0, v_1, v_2) , we solve $p(t) = v_0 + \beta(v_1 - v_0) + \gamma(v_2 - v_0)$ for the unknowns (t, β, γ) . The 3x3 linear system is solved via Cramer's rule using scalar triple products with the edge vectors $e_1 = v_1 - v_0$ and $e_2 = v_2 - v_0$. The intersection is valid iff $\beta \geq 0$, $\gamma \geq 0$, $\beta + \gamma \leq 1$, and $t > \epsilon$. The barycentric coordinates (β, γ) are kept for later UV interpolation.

3.2 Camera ray generation

The camera basis is computed once at scene load: $w = -\text{gaze}$, $u = \text{up} \times w$, $v = w \times u$. For pixel (i, j) (with $j = 0$ at the top), the world-space pixel position is

```

m = eye + (-w) * near_distance      (image plane center)
q = m + left * u + top * v          (top-left corner)
s = q + s_u * u - s_v * v           (pixel center)

```

where s_u, s_v are the per-pixel offsets. The ray direction is `normalize(s - eye)`. This handles arbitrary cameras correctly, including when the user-supplied `up` is not perfectly perpendicular to `gaze` — the cross-product reconstruction makes the basis orthonormal.

3.3 Phong shading

Surface color is $\text{ambient} + \sum (\text{diffuse} + \text{specular})$. The ambient term is $M_{\text{ambient}} \cdot L_{\text{ambient}}$. For each unblocked light, the diffuse term is $M_{\text{diffuse}} \cdot (I/r^2) \cdot \max(0, N \cdot L)$ (Lambert with inverse-square falloff) and the specular term is $M_{\text{specular}} \cdot$

$(I/r^2) \cdot \max(0, N \cdot H)^p$ where $H = \text{normalize}(L + V)$ is the Blinn-Phong halfway vector and p is the material's **phongexponent**.

3.4 Triangular lights

Per the spec, the emission direction is $(v_1 - v_2) \times (v_1 - v_3)$. The light vector L reaching a surface point is the negation of this direction; the shadow target and distance attenuation use the triangle's centroid.

3.5 Shadows

For each (point, light) pair, a shadow ray is cast from the hit point — offset by $\epsilon \cdot N$ along the surface normal — toward the light. If any triangle is hit at distance less than the light distance (minus the same ϵ), the light's contribution is skipped.

The constant **Scene::SHADOW_EPSILON = 1e-3f** was chosen experimentally: smaller values produce visible “shadow acne” (self-shadowing speckles caused by floating-point error in the intersection routine); larger values cause shadows to detach from their casters. The assignment explicitly required defining this constant.

3.6 Recursive mirror reflection

If a hit material has nonzero **mirrorreflectance** and the current ray's depth is less than **maxraytracedepth**, a reflected ray is cast. The reflection direction is $R = D - 2(D \cdot N)N$. The reflected ray's color is added to the surface color, weighted component-wise by the material's mirror coefficient. The depth counter prevents infinite recursion in mutual-mirror scenes (which would otherwise exhaust the stack).

3.7 Texture sampling

Per-vertex UV coordinates are interpolated at the hit point using the same barycentric weights produced by Möller-Trumbore: $uv = (1 - \beta - \gamma) \cdot uv_0 + \beta \cdot uv_1 + \gamma \cdot uv_2$. The sampler converts (u, v) to pixel coordinates $(u \cdot (W - 1), (1 - v) \cdot (H - 1))$ — with vertical flip because image row 0 is at the top while UV $v = 0$ is at the bottom — wraps out-of-range UVs with **floor**, and returns the texel as RGB. The final color blend is $(1 - \text{texturefactor}) \cdot \text{shaded} + \text{texturefactor} \cdot \text{texel}$, matching the spec's definition: **texturefactor=0** means texture has no effect, **texturefactor=1** means texture color is used directly without shading.

Per the assignment, only one texture image per scene is supported (**<textureimage>** is a single tag, “addressed by all texture coordinates”).

4. Acceleration: Bounding Volume Hierarchy (+20)

A linear search over the 18,192 triangles in our test scene would require ~117 billion intersection tests at 800x800 resolution — and that’s before accounting for shadow and reflection rays, which can multiply the cost by 5-10x.

We implemented a **top-down median-split BVH**:

1. Each triangle is wrapped in an axis-aligned bounding box (AABB) and assigned a centroid.
2. Recursively: if a node contains ≤ 4 triangles, make it a leaf. Otherwise, find the AABB of all centroids, pick the longest axis, and split at the median centroid using `std::nth_element` (an $O(N)$ partial sort — much cheaper than a full sort).
3. Internal nodes store left and right child indices; leaves store a `[first, count)` range into a flat triangle-reference array.

Traversal is **iterative with a fixed-size stack of 64 entries** to avoid recursion overhead. Each node’s AABB is tested against the ray using the standard slab method — for each axis, compute the entry/exit t values from the box’s near and far planes, swap if the ray direction is negative on that axis, then check whether the resulting interval is non-empty and overlaps $[t_{\min}, t_{\text{current_closest}}]$. Subtrees whose AABB the ray misses are skipped entirely.

For our 18,192-triangle scene, this reduces the average per-ray triangle test count from 18,192 to roughly 30 — a ~600x reduction. The BVH itself builds in only a few milliseconds at scene load, so the cost is amortized over millions of rays.

We chose median split over the Surface Area Heuristic (SAH). SAH produces slightly better trees but is significantly more code and not needed at this scene size — median split already gives sub-100 ms render times.

The same BVH is used for shadow rays in `inShadow()`, with one optimization: shadow rays don’t need the *closest* hit, just *any* hit closer than the light, so we short-circuit on first hit.

5. Multithreading (+10)

Ray tracing is embarrassingly parallel: each pixel’s color depends only on read-only scene data. We exploit this with a **`std::thread` worker pool using dynamic row-level scheduling**:

```
std::atomic<int> next_row(0);
auto worker = [&]() {
    while (true) {
        int j = next_row.fetch_add(1);
        if (j >= H) break;
        for (int i = 0; i < W; ++i) {
```

```

        // shade pixel (i, j)
    }
}
};

```

`std::atomic<int>::fetch_add` atomically returns the current value and increments. Whichever thread reaches the line first claims the next row. This is **dynamic scheduling** — it automatically balances load even when some rows are heavier than others (a row containing the reflective floor will trigger more recursive rays than a row showing only background).

We chose `std::thread` over OpenMP because Apple’s Clang on macOS does not bundle the OpenMP runtime, and `std::thread` is a portable C++17 feature with no extra build configuration. The implementation is also more transparent: every line of thread management is visible in the source, which made debugging the worker logic straightforward.

The image buffer `img` is a single contiguous `std::vector<unsigned char>`. Each thread writes to disjoint byte ranges (different rows = different pixel index ranges), so **no synchronization is needed on the buffer**. This is one of the rare cases where lock-free parallelism is the natural choice rather than an optimization.

We used `std::thread::hardware_concurrency()` to size the pool to the number of logical cores available (8 on the test machine).

6. Benchmarks

All measurements are on the standing-girl test scene: **18,192 triangles, 2 meshes, 2 materials, 2 point lights**. Image size 800x800. Test machine: MacBook (8 logical cores), Apple Clang 17, -O3. Each timing is the wall-clock time of the rendering loop only, excluding scene load and BVH build.

6.1 Comparison table

Configuration	Render time	Speedup vs. baseline
Naive + 1 thread	50,065 ms	1.0x (baseline)
Naive + 8 threads	11,502 ms	4.4x
BVH + 1 thread	367 ms	136x
BVH + 8 threads	81 ms	618x

6.2 Discussion

BVH dominates threading. The acceleration structure alone gives a 136x speedup; adding 8 threads on top adds another ~4.5x to reach 618x. This

matches the lecture-slide observation that a 70k-triangle bunny renders in 2 seconds with a spatial structure and ~2 hours without.

Threading speedup is sub-linear (4.4x on 8 threads). Three causes:

1. **Asymmetric cores.** Modern Mac CPUs mix performance and efficiency cores; the efficiency cores run at lower clock and reach saturation faster, so the marginal benefit of using all 8 cores is less than 8x.
2. **Memory bandwidth and BVH pointer-chasing.** All threads read the same scene data simultaneously. The working set fits in cache, but tree traversal is inherently latency-bound, which is harder to parallelize cleanly.
3. **Amdahl's law.** The single-threaded portion (BVH build, scene parse, image write) is small but non-zero. As the parallel section gets faster, the serial section becomes proportionally more visible.

The combined speedup (618x) is roughly multiplicative of the two individual speedups ($136 * 4.5 \sim 612$). This indicates BVH and threading attack independent bottlenecks: BVH reduces work-per-ray; threading distributes that reduced work across cores.

At 3200x3200 the BVH+threads configuration takes ~1.4 s for the same scene, scaling roughly linearly with pixel count. The naive single-threaded configuration would take roughly $50,065 * 16 \sim 13$ minutes at that resolution, which we did not measure exhaustively but estimated.

7. Sample Renders

7.1 Final hero image (3200x3200, BVH + 8 threads)

The girl is rendered with material textured by the imported skin base map; the floor uses a checkered texture combined with mirror reflectance, producing the visible reflection of the figure with the checkered pattern still showing through.

7.2 Untextured render at high resolution

Same scene with `texturefactor = 0` on both materials. Useful for clean visualization of geometry, shading, and shadows.

7.3 Benchmark-resolution render (800x800)

The same scene at the resolution used for all timing measurements.

7.4 Mirror sphere — early development scene

A subdivided icosahedron used during development to visually verify that mirror reflection, point and triangular light shading, and shadow casting all worked

together.

7.5 First successful mirror reflection

The first time recursive reflection produced a visible mirror image of the red triangle on the gray floor — a development milestone.

7.6 First raster output

The very first image produced: white pixels where rays hit the test triangle, black background. No shading, no mirror, no shadow yet — but the camera-ray and intersection plumbing was working end-to-end. This was the foundation everything else was built on top of.

8. Learning Process

The development order was deliberately incremental: each phase produced a visible result before the next was started, so any bug could be localized to the most recent change.

Math types first (`Vec3`, `Ray`). Trivial in isolation but every later component depends on them being correct.

Scene loader. TinyXML-2 was straightforward; the trickiest part was parsing the OBJ-style `v/t/n` face tokens (handling `v`, `v//n`, `v/t`, and `v/t/n` cases) and converting 1-based to 0-based indices. The first iteration had off-by-one errors in face indexing that produced a black image — easy to diagnose because `traceClosest` returned no hits.

Camera rays. The first non-black image had the test triangle visible, but the *first* version of the basis math had a sign error — `w` was equal to `gaze` instead of `-gaze`, sending all rays backward. Debugging required adding a print of the center-pixel ray direction.

Möller-Trumbore. Worked first try. The sign convention for `t` (positive = in front of camera) caught me only because I forgot the `t >  $\epsilon$` check briefly, which produced rays hitting “themselves” at `t = 0`.

Phong shading. The first lit image was nearly black because I assumed light intensities were in `[0,1]` but the assignment uses `[0,255]`-style raw values. The fix was conceptual (no clamping changes), just turning up the test light intensity and confirming the falloff with $1/r^2$ was correct.

Shadows. The classic shadow-acne bug appeared on the first run because I forgot the `$\epsilon \cdot N$` offset on the shadow ray origin. With the offset added (`SHADOW_EPSILON = 1e-3`), the speckles disappeared and shadows looked clean.

Mirror reflection. Worked on first try once `rayColor` was made recursive. The `depth < max_depth` check is essential — without it, two parallel mirrors would crash with stack overflow.

BVH. The trickiest part was a subtle bug where I held a reference to a vector element across an `emplace_back`, which can invalidate it when the vector reallocates. The fix was to write to a local first and copy after recursion. The slab test for ray-AABB intersection needed careful sign handling for negative ray directions.

Multithreading. Worked almost immediately after I removed leftover `static int debug_counter` variables from earlier debug sessions — those were data races. The atomic row counter was the right primitive: simple, contention-free in the common case, and load-balancing for free.

Textures. Took longer than expected because the imported model was multi-material (separate textures for face, body, top, bottom) but the spec only supports one global texture per scene. Resolution was to demonstrate texturing on the floor (with a procedurally-generated checker PNG) while leaving the figure with its body skin tone applied via the same single-texture mechanism.

A consistent theme throughout: **add one feature, see one new visual artifact, fix it, move on.** This is far less risky than implementing everything at once and trying to debug a fully-broken renderer.

9. Build and Run

See `README.md` for full instructions. Brief version:

`make`

`./raytracer scenes/standing_girl.xml output.png`

CLI flags `--no-bvh` and `--no-threads` were added for benchmarking and disable the respective bonus feature.

10. Bonuses Claimed

- **+20** Hierarchical acceleration structure (BVH) — `include/BVH.h`, `src/BVH.cpp`. 136x speedup at 18,192 triangles.
- **+10** Multithreading — `src/Renderer.cpp` (atomic-counter row pool with `std::thread`). Additional 4.4x on 8 cores.

Combined: ~620x speedup over the naive single-threaded baseline.